LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 4 TENATANG QUEUE

Disusun Oleh :

M.FAJAR FADHILUL ZIKRI

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, TAHUN 2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya, sehingga laporan praktikum ini dapat diselesaikan tepat pada waktunya. Laporan ini disusun sebagai dokumentasi dan evaluasi dari materi mata kuliah Struktur Data, yang berfokus pada cara mengorganisasi serta mengelola data secara logis dalam sistem komputer.

Praktikum ini memberikan pemahaman mengenai pentingnya pemilihan metode penyimpanan data yang tepat. Struktur data bukan sekadar cara menyimpan informasi, melainkan fondasi dalam membangun algoritma yang efisien. Dengan pengorganisasian data yang baik, sebuah program dapat memproses informasi dengan kecepatan yang lebih tinggi dan penggunaan memori yang lebih optimal.

Melalui rangkaian praktikum ini, mahasiswa diharapkan tidak hanya menguasai teori pengolahan data, tetapi juga mampu mengimplementasikannya dalam bentuk kode program yang sistematis. Kemampuan untuk menyusun data secara terstruktur merupakan keterampilan krusial yang dibutuhkan untuk memecahkan masalah komputasi yang lebih kompleks di masa mendatang.

Terima kasih kami sampaikan kepada dosen pembimbing atas bimbingan dan ilmu yang diberikan selama proses pembelajaran. Terima kasih juga kepada rekan-rekan mahasiswa yang telah memberikan dukungan serta berbagi ide selama praktikum berlangsung. Penyusun menyadari bahwa laporan ini masih memiliki ruang untuk perbaikan, sehingga kritik dan saran yang membangun akan sangat dihargai.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan membantu dalam memahami betapa pentingnya peran struktur data dalam dunia pengembangan perangkat lunak.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR……ii

DAFTAR ISI……..iii

BAB 1 PENDAHULUAN……..……1

1.1 Pengertian Praktikum……1

1.2 Tujuan Praktikum……..1

1.3 Persyaratan Praktikum…….……..1

1.4 Tempat dan Waktu Praktikum…….……..2

BAB 2 PEMBAHASAN PRAKTIKUM…….3

2.1 Pengertian Queue…….3

2.2 Langkah-Langkah Praktikum…….3

BAB 3 PENUTUP……..……15

3.1 Kesimpulan……...……15

LAMPIRAN……...……16

BAB 1
PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan cara pengorganisasian, penyimpanan, dan pengelolaan data secara efisien.

Kegiatan ini tidak hanya menekankan pada penguasaan teori mengenai model-model data, tetapi juga pada latihan implementasi struktur penyimpanan yang optimal, analisis efisiensi algoritma, hingga pengujian performa program. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual mengenai tata kelola data dengan kemampuan teknis dalam memecahkan masalah komputasi yang kompleks secara sistematis.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Struktur data.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
6. Mengetahui dan mengaplikasikan Tipe Data stack, queue dll.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.

2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.

3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.

4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.

5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum struktur data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Queue

1. Definisi Umum

Queue atau antrean adalah salah satu struktur data linier yang bekerja berdasarkan prinsip FIFO (First In, First Out). Dalam prinsip ini, data yang pertama kali masuk ke dalam struktur akan menjadi data yang pertama kali diproses atau dikeluarkan. Konsep ini identik dengan antrean manusia di dunia nyata, di mana orang yang datang pertama akan dilayani terlebih dahulu.

2. Komponen Utama

Dalam pengoperasiannya, Queue memiliki dua variabel penunjuk (pointer) utama:

- Front (Head): Bagian depan antrean yang digunakan sebagai pintu keluar bagi elemen yang akan dihapus.
- Rear (Tail): Bagian belakang antrean yang digunakan sebagai pintu masuk bagi elemen baru.

3. Fungsi dan Operasi Dasar

Berikut adalah fungsi-fungsi standar yang digunakan untuk memanipulasi data dalam Queue:

- Enqueue: Operasi untuk menambahkan atau memasukkan elemen baru ke posisi paling belakang (Rear) dari antrean.
- Dequeue: Operasi untuk menghapus atau mengeluarkan elemen dari posisi paling depan (Front) antrean.
- Peek (Front): Operasi untuk melihat atau mengambil nilai dari elemen yang berada di posisi terdepan tanpa menghapusnya.
- isEmpty: Fungsi untuk memeriksa apakah antrean sedang dalam keadaan kosong. Ini penting dilakukan sebelum menjalankan operasi Dequeue untuk menghindari error.
- isFull: Fungsi untuk memeriksa apakah antrean sudah mencapai kapasitas maksimalnya (biasanya digunakan pada implementasi berbasis Array statis).
- Size: Fungsi untuk menghitung dan mengembalikan jumlah total elemen yang saat ini tersimpan di dalam antrean.

4. Karakteristik Queue

- Akses Terbatas: Kita tidak diperbolehkan mengambil atau menyisipkan data di tengah-tengah antrean. Semua akses harus melalui Front atau Rear.
- Urutan Konsisten: Urutan data dijamin tetap sama dari saat masuk hingga saat keluar.
- Fleksibilitas Implementasi: Queue dapat dibuat menggunakan Array jika ukurannya sudah diketahui (statis), atau menggunakan Linked List jika membutuhkan ukuran yang bisa berubah-ubah (dinamis).

5. Contoh Penerapan dalam Teknologi

Queue digunakan secara luas dalam sistem komputer, di antaranya:

- CPU Scheduling: Mengatur urutan proses yang mengantre untuk dieksekusi oleh prosesor.
- Printer Spooling: Mengelola urutan dokumen yang masuk ke memori printer agar dicetak sesuai urutan pengiriman.
- Traffic Management: Mengatur aliran paket data dalam jaringan internet untuk mencegah penumpukan data.

2.7 Langkah Langkah Praktikum

Pada praktikum kali ini kita akan membuat 5 buah class yang pertama Adalah **QueueArray:**

```
public class QueueArray_2511533023 {
    int front, rear, size;
    int capacity;
    int array[];

    public QueueArray_2511533023(int capacity) {
        this.capacity = capacity;
        front = this.size = 0;
        rear = capacity - 1;
        array = new int[this.capacity];
    }

    boolean isFull_3023(QueueArray_2511533023 queue) {
        return (queue.size == queue.capacity);
    }

    boolean isEmpty_3023(QueueArray_2511533023 queue) {
        return (queue.size == 0);
    }

    void enqueue_3023(int item) {
        if (isFull_3023(this)) {
            System.out.println("Queue is full");
            return;
        }
        this.rear = (this.rear + 1) % this.capacity;
        this.array[this.rear] = item;
        this.size = this.size + 1;
        System.out.println(item + " enqueued to queue");
    }
    int dequeue_3023() {
        if (isEmpty_3023(this)) {
            return Integer.MIN_VALUE;
        }
        int item = this.array[this.front];
        this.front = (this.front + 1) % this.capacity;
        this.size = this.size - 1;
        return item;
    }

    int front_3023() {
        if (isEmpty_3023(this)) {
            return Integer.MIN_VALUE;
```

```
        }
        return this.array[this.front];
    }

    int rear_3023() {
        if (isEmpty_3023(this)) {
            return Integer.MIN_VALUE;
        }
        return this.array[this.rear];
    }

    // mencetak elemen antrian
    void display_3023() {
        int i;
        if (front == rear) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        // kunjungi dari belakang dan cetak
        for (i = front; i < rear; i++) {
            System.out.printf(" %d <-- ", array[i]);
        }
        return;
    }
}
```

Hasil Output:

Tidak ada karena ini merupakan class blueprint atau kerangka saja yang nanti hasil nya dapat di panggil pakai class driver.

Class ke-2 adalah QueueArrayDriver:

```
public class QueueArrayDriver_2511533023 {
    public static void main(String[] args) {
        // membuat objek queue dengan kapasitas maksimal 1000 elemen
        QueueArray_2511533023 queue = new QueueArray_2511533023(1000);

        // menambahkan beberapa data ke dalam antrean
        queue.enqueue_3023(10);
        queue.enqueue_3023(20);
        queue.enqueue_3023(30);
        queue.enqueue_3023(40);

        // menampilkan elemen yang saat ini berada di urutan paling depan dan belakang
        System.out.println("Item di depan " + queue.front_3023());
        System.out.println("item paling belakang " + queue.rear_3023());

        // menampilkan seluruh isi antrean saat ini
        System.out.println("tampilan queue");
        queue.display_3023();
        System.out.println();

        // menghapus satu elemen terdepan (FIFO) dan menampilkannya
        System.out.println(queue.dequeue_3023() + " dihapus dari queue");

        // menampilkan status posisi depan dan belakang setelah penghapusan
        System.out.println("item di depan:" + queue.front_3023());
        System.out.println("item dibelakang: " + queue.rear_3023());

        // menampilkan isi antrean terbaru setelah satu data dihapus
        System.out.println("tampilan queue setelah satu data dihapus");
        queue.display_3023();
    }
}Hasil Output:
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
item paling belakang 40
tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
item di depan:20
item dibelakang: 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

Class ke-3 IterasiQueue:

```
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511533023 {
    public static void main(String[] args) {
        // membuat objek queue menggunakan LinkedList
        Queue<String> q_3023 = new LinkedList<>();

        // menambahkan elemen-elemen ke dalam queue
        q_3023.add("Praktikum");
        q_3023.add("Struktur");
        q_3023.add("Data");
        q_3023.add("Dan");
        q_3023.add("Algoritma");

        // membuat objek iterator untuk menelusuri elemen queue
        Iterator<String> iterator = q_3023.iterator();

        // melakukan perulangan selama masih ada data di dalam iterator
        while (iterator.hasNext()) {
            // mencetak elemen dan berpindah ke data selanjutnya
            System.out.print(iterator.next() + " ");
        }
    }
}
```

Hasil Output:
Praktikum Struktur Data Dan Algoritma

Mempelajari cara mengakses atau menelusuri elemen di dalam Queue tanpa harus menghapusnya satu per satu. Dengan menggunakan objek **Iterator**, program ini menunjukkan teknik melintasi (*traversing*) sekumpulan data dalam koleksi Queue secara berurutan dari depan ke belakang untuk ditampilkan ke layar

Class ke-4 ReverseData:

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511533023 {
    public static void main(String[] args) {
        // inisialisasi queue menggunakan implementasi linkedlist
        Queue<Integer> q_3023 = new LinkedList<Integer>();
```

```
        // menambahkan data ke dalam queue
        q_3023.add(1);
        q_3023.add(2);
        q_3023.add(3); // [1, 2, 3]

        // menampilkan isi queue asli sebelum proses balik (reverse)
        System.out.println("sebelum reverse" + q_3023);

        // inisialisasi stack untuk membantu proses pembalikan data
        Stack<Integer> s_3023 = new Stack<Integer>();

        // Proses Membalikkan Data: Memindahkan semua elemen dari Queue ke dalam
Stack
        while (!q_3023.isEmpty()) {
            // menghapus elemen dari queue dan memasukkannya ke stack (LIFO)
            s_3023.push(q_3023.remove());
        }

        // Proses Mengembalikan Data: Memindahkan kembali semua elemen dari Stack
ke Queue
        while (!s_3023.isEmpty()) {
            // mengambil elemen dari stack (paling atas) dan memasukkannya kembali ke
queue
            q_3023.add(s_3023.pop());
        }

        // menampilkan hasil akhir queue yang datanya sudah terbalik
        System.out.println("sesudah reverse= " + q_3023); // [3, 2, 1]
    }
}
```

Hasil Output:
sebelum reverse[1, 2, 3]
sesudah reverse= [3, 2, 1]

Mempelajari interaksi antar dua struktur data yang berbeda (Queue dan Stack). Program ini memiliki tujuan logika yang spesifik: membalikkan urutan data. Karena Queue bersifat FIFO (masuk pertama, keluar pertama) dan Stack bersifat LIFO (masuk terakhir, keluar pertama), memindahkan data dari Queue ke Stack lalu dikembalikan lagi akan secara otomatis membalikkan urutan data tersebut.

Class ke-5 QueueLinkedList:

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511533023 {
    public static void main(String[] args) {
        // Menggunakan akhiran _3023 pada nama objek queue
        Queue<Integer> q_3023 = new LinkedList<>();

        // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
        for (int i = 0; i < 6; i++) {
            q_3023.add(i);
        }

        // Menampilkan isi antrian
        System.out.println("Elemen Antrian " + q_3023);

        // Untuk menghapus kepala antrian
        int hapus_3023 = q_3023.remove();
        System.out.println("Hapus elemen = " + hapus_3023);
        System.out.println(q_3023);

        // Untuk melihat antrian terdepan
        int depan_3023 = q_3023.peek();
        System.out.println("Kepala Antrian = " + depan_3023);

        int banyak_3023 = q_3023.size();
        System.out.println("Size Antrian = " + banyak_3023);
    }
}
```

Hasil Output:

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Mengimplementasikan Queue menggunakan pustaka standar Java (*Java Collection Framework*) berbasis **LinkedList**. Berbeda dengan program pertama yang menggunakan Array manual, program ini bertujuan menunjukkan cara praktis menggunakan Queue yang sudah disediakan oleh Java. Ini mengajarkan Anda bahwa Queue dalam Java sering kali diimplementasikan melalui kelas LinkedList karena sifatnya yang dinamis (ukurannya bisa bertambah secara fleksibel).

BAB 3
PENUTUP

3.1 Kesimpulan

dari praktikum ini saya menjadi lebih memahami bahwa *Queue* merupakan struktur data yang sangat berguna untuk mengatur urutan data dengan prinsip FIFO, atau pertama masuk pertama keluar. Saya mempelajari bahwa implementasi *Queue* bisa dilakukan secara manual menggunakan *Array* dengan memanfaatkan indeks *front* dan *rear*, maupun secara praktis menggunakan kelas *LinkedList* yang sudah tersedia di Java. Selain itu, saya juga memahami cara melakukan iterasi untuk mengecek isi antrean serta teknik membalikkan urutan data dengan bantuan *Stack*. Secara keseluruhan, praktikum ini memberikan gambaran jelas bagi saya mengenai bagaimana sistem komputer mengelola antrean tugas atau proses secara teratur dan sistematis.

Tugas Queue:

Class pertama AntrianLoket_2511533023:

```
public class AntrianLoket_2511533023 {
   // Variabel wajib sesuai standar tugas dengan tambahan _3023
   private int front_3023; // indeks depan
   private int rear_3023;  // indeks belakang
   private int max_3023;   // kapasitas array
   private String[] queue_3023; // array penyimpan data

   // Inisialisasi awal dan deklarasi kapasitas
   public AntrianLoket_2511533023(int kapasitas) {
      this.max_3023 = kapasitas;
      this.queue_3023 = new String[max_3023];
      this.front_3023 = -1; // penanda antrean kosong
      this.rear_3023 = -1;  // penanda antrean kosong
   }

   // Method untuk mengecek apakah antrean kosong
   public boolean isEmpty_3023() {
      return (front_3023 == -1); // mengembalikan true jika kosong
   }

   // Method untuk mengecek apakah antrean penuh
   public boolean isFull_3023() {
      return (rear_3023 == max_3023 - 1); // mengembalikan true jika penuh
   }

   // Method untuk menambah data ke belakang antrian
   public void enqueue_3023(String nama) {
      if (isFull_3023()) {
         System.out.println("Antrean penuh! Tidak dapat menambah pelanggan."); //
pesan jika penuh
      } else {
         if (isEmpty_3023()) {
            front_3023 = 0;
         }
         rear_3023++;
         queue_3023[rear_3023] = nama;
         System.out.println("Data berhasil ditambahkan ke antrian"); // pesan sukses
      }
   }

   // Method untuk menghapus data dari depan antrian
   public String dequeue_3023() {
      if (isEmpty_3023()) {
         System.out.println("Antrean kosong!"); // pesan jika kosong
```

```
            return null;
        } else {
            String nama = queue_3023[front_3023];
            // Menggeser data untuk mempertahankan prinsip antrean array linear
            for (int i = 0; i < rear_3023; i++) {
                queue_3023[i] = queue_3023[i + 1];
            }
            if (front_3023 == rear_3023) {
                front_3023 = -1;
                rear_3023 = -1;
            } else {
                rear_3023--;
            }
            return nama;
        }
    }

    // Method untuk menampilkan seluruh isi antrian
    public void display_3023() {
        if (isEmpty_3023()) {
            System.out.println("Antrean Kosong");
        } else {
            System.out.println("Isi antrian:"); // header isi antrean
            for (int i = 0; i <= rear_3023; i++) {
                System.out.println((i + 1) + ". " + queue_3023[i]); // format list antrean
            }
        }
    }

    // Method untuk membalik seluruh isi antrian
    public void Reverse_3023() {
        if (isEmpty_3023()) return;
        java.util.Stack<String> s = new java.util.Stack<>();
        while (!isEmpty_3023()) {
            s.push(dequeue_3023());
        }
        while (!s.isEmpty()) {
            enqueue_3023(s.pop());
        }
    }
}
```

Class ini berfungsi sebagai kerangka / blueprint yang akan dikendalikan oleh class Queue sebagai driver.

Class kedua Queue_2511533023:

```
import java.util.Scanner;

public class Queue_2511533023 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Kapasitas maksimal queue adalah 10
        AntrianLoket_2511533023 loket_3023 = new AntrianLoket_2511533023(10);
        int pilih_3023;

        do {
            // Menampilkan menu utama antrean loket
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");

            pilih_3023 = sc.nextInt();
            sc.nextLine();

            switch (pilih_3023) {
                case 1:
                    // Proses enqueue untuk menambah pelanggan
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama_3023 = sc.nextLine();
                    loket_3023.enqueue_3023(nama_3023);
                    break;
                case 2:
                    // Proses dequeue untuk menghapus pelanggan terdepan
                    String dilayani_3023 = loket_3023.dequeue_3023();
                    if (dilayani_3023 != null) {
                        System.out.println(dilayani_3023 + " telah dilayani");
                    }
                    break;
                case 3:
                    // Menampilkan seluruh isi antrean saat ini
                    loket_3023.display_3023();
                    break;
                case 4:
                    // Membalikkan urutan antrean
                    loket_3023.Reverse_3023();
                    break;
                case 5:
```

```
                // Mengakhiri program
                System.out.println("Program selesai");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
                break;
        }
    } while (pilih_3023 != 5);

    sc.close();
  }
}
```

Hasil Output:

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: joko
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: jack
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. joko
2. jack
```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. jack
2. joko

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
jack telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. joko

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. KeluarPilih menu: 5
Program selesai

Fungsi dari program ini Adalah memanggil atribut yang sudah kita bikin di class website yang di mana itu merupakan kerangka /blueprint untuk program ini.